Backtesting Lab & Quant Lab — Formula Reference

Every number the Backtesting Lab and the Quant Lab display, grouped exactly as the UI groups it, with the formula the engine actually evaluates and a worked example for each.

Scope. Backtesting Lab (`/backtesting`) and Quant Lab (`/quant-lab`) only. Portfolio, Markets, AI Analyzer and order execution are out of scope.

How to read an entry. Every metric gets: the label as it appears on screen, the formula, the source file and line that computes it, a worked example carried through arithmetic, and any behaviour that would surprise you. Where a metric appears in more than one tab it is listed in **each** tab — the second listing states the formula reference rather than repeating the derivation, and calls out any difference in formatting or edge-case handling.

Verification. Every numeric result below was produced by executing the repository's own functions, not by hand arithmetic. Reproduce them by importing from `@/utils/backtesting`, `@/utils/quantLab`, `@/calculations/*`.

Table of contents

---

§0 — Shared foundations

Everything in Parts I and II is built from this handful of primitives. They are documented once here and referenced by number throughout.

0.1 Notation

Symbol	Meaning
P_t	Close price at bar t
r_t	Simple return at bar t
V_t	Portfolio (equity) value at bar t
R_p, R_m	Portfolio / benchmark return series
R_f	Annual risk-free rate
P	**Periods per year** — inferred, not assumed (see §0.3)
w	Weight vector, $\sum_i w_i = 1$, $w_i \geq 0$
μ	Vector of annualized expected returns
Σ	Annualized covariance matrix
n	Number of assets
T	Number of observations in the estimation window

0.2 Price alignment

Code: `src/utils/backtesting.ts:754` — `alignPriceSeries()`

Before anything is computed, every selected asset **and the benchmark** are intersected onto the set of dates they *all* have a price for, then trimmed to the requested period.

$$\mathcal{D} = \bigcap_i \text{dates}(S_i), \qquad \mathcal{D}_{\text{used}} = \{d \in \mathcal{D} : d \geq d_{\max} - \text{days}\}$$

Example. BTC trades 365 days a year; AAPL trades ~252. A BTC + AAPL portfolio runs on the **intersection** — the equity calendar — so the crypto sleeve contributes ~252 observations, not 365. This is deliberate: you cannot rebalance a stock on a Sunday.

Consequence you will see in the UI. Requesting 5Y with an asset that only has 8 months of history yields 8 months. `coverage.truncated` fires and the banner names the limiting symbol, rather than silently drawing a shorter chart.

0.3 Periods per year (annualization basis)

Code: `src/calculations/shared.ts:170` — `inferPeriodsPerYear()`

$$P = \frac{365.25}{\overline{\Delta d}}, \qquad \overline{\Delta d} = \frac{d_{\text{last}} - d_{\text{first}}}{N - 1} \ \text{(in days)}$$

This is not hardcoded to 252. Hardcoding 252 would understate crypto volatility by a factor of $\sqrt{365/252} \approx 1.20$.

Example.

Series	Avg gap	P
Weekday equity (Mon–Fri)	1.40 d	**260.89**
Continuous crypto	1.00 d	**365.25**
Mixed BTC + AAPL (intersected)	1.40 d	**≈ 260.89**

0.4 Simple return

Code: `src/calculations/shared.ts:24` — `calculateReturns()`

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Example. $P_0 = 10{,}000$, $P_1 = 10{,}200 \rightarrow r_1 = 200/10{,}000 = \mathbf{0.0200}$ (+2.00%).

An N-price series produces $N - 1$ returns. This one-element offset is why the rolling charts plot against `dates[i+1]`.

0.5 Log return

Code: `src/calculations/shared.ts:32` — `calculateLogReturns()`

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Example. $\ln(10{,}200/10{,}000) = \ln(1.02) = \mathbf{0.019803}$ — slightly below the simple return 0.020000, as always for positive returns.

Exported and available; the engine's headline metrics use **simple** returns.

0.6 Mean and standard deviation

Code: `src/calculations/shared.ts:1` and `:5`

$$\bar{x} = \frac{1}{n}\sum_i x_i, \qquad s = \sqrt{\frac{1}{n-1}\sum_i (x_i - \bar{x})^2}$$

Standard deviation uses the **sample** convention ($n-1$), matching the diagonal of the covariance matrix in §I.5.16. Returns 0 for $n < 2$ rather than dividing by zero.

Example (Dataset E, Appendix A). $\bar{r} = \mathbf{0.001996}$, $s = \mathbf{0.029386}$.

0.7 Empirical quantile (Hyndman–Fan type 7)

Code: `src/calculations/shared.ts:51` — `quantile()`

Sort ascending, then interpolate linearly:

$$h = (n-1)\,q, \qquad Q(q) = x_{\lfloor h \rfloor} + (h - \lfloor h \rfloor)\left(x_{\lceil h \rceil} - x_{\lfloor h \rfloor}\right)$$

This is `numpy.quantile`'s default method, chosen so the published Python reference agrees with the TypeScript engine by construction rather than coincidence.

Example. Dataset E's 10 returns sorted:

```
-0.040035, -0.029994, -0.020058, -0.020049, -0.010000,
 0.020000,  0.020026,  0.020050,  0.040008,  0.040016
```

For $q = 0.05$: $h = 9 \times 0.05 = 0.45$, so $Q = -0.040035 + 0.45 \times (-0.029994 - (-0.040035)) = -0.040035 + 0.004518 = \mathbf{-0.035517}$.

> **Why interpolation matters.** A naive $\lfloor q(n-1) \rfloor$ index returns -0.040035 — the single worst observation — as the 5% VaR regardless of how the rest of the tail is shaped. On short samples that is severely biased.

0.8 Inverse normal CDF and normal PDF

Code: `src/calculations/shared.ts:70` — `normInv()` (Acklam's rational approximation, $|\varepsilon| < 1.15 \times 10^{-9}$); `:118` — `normPdf()`

$$z_\alpha = \Phi^{-1}(\alpha), \qquad \phi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$$

Example.

α	z_α	$\phi(z_\alpha)$
0.90	1.281552	0.175498
0.95	1.644854	0.103136
0.99	2.326348	0.026652

Continuous in α — the confidence slider in Quant Lab moves through every value from 90% to 99%, not four bucketed z-scores.

0.9 Seeded RNG and Gaussian draw

Code: `src/calculations/shared.ts:122` — `mulberry32()`; `:132` — `gauss()` (Box–Muller)

$$z = \sqrt{-2 \ln u_1}\, \cos(2\pi u_2), \qquad u_1, u_2 \sim \mathcal{U}(0,1)$$

Used only by Monte Carlo (§II.1). The seed is fixed at 42, so the same configuration always produces the same paths.

Part I — Backtesting Lab

I.0 — Backtest configuration

These inputs are not displayed as results, but every formula below consumes them, so they are defined here.

Control	Symbol	Default	Notes
Asset universe + weights	w^{user}	AAPL 40 / BTC 40 / BICB 20	Must total 100%
Period	days	1 Y = 365	3M=91, 6M=182, 1Y=365, 3Y=1095, 5Y=1825
Initial capital	V_0	10,000	
Strategy	—	Buy & Hold	See §I.5.12–§I.5.15
Rebalance	—	Monthly	Calendar buckets, not fixed bar counts
Benchmark	R_m	BTC	Also intersected in §0.2
Fees (bps)	f	10 → 0.0010	
Slippage (bps)	s	5 → 0.0005	
Risk-free (%)	R_f	0.0	Annual

I.0.1 Estimation window (lookback)

Code: `src/utils/backtesting.ts:70` — `resolveLookback()`

$$L = \min\left(252,\ \max\left(20,\ \left\lfloor \frac{N}{3} \right\rfloor\right)\right)$$

One third of the series is surrendered to the estimation window, floored at 20 observations and capped at 252.

Example.

Aligned bars N	Lookback L
45	20 (floor binds)
60	20 (floor binds)
300	100
800	252 (cap binds)
1250	252 (cap binds)

This window is consumed before the first allocation exists, so it never appears on the equity curve. That is why a "1 Y" backtest ending 2026-08-17 starts its chart around late December 2025 rather than 2025-08-17 — roughly 83 trading days of warm-up on a ~250-bar series. The header states it: *"estimation window: L bars, R rebalances, δ% shrinkage."*

I.0.2 Look-ahead guarantee

At every rebalance date the engine estimates from `returns[max(0, absolute − L) : absolute]` — the slice **excludes** the current bar. No weight is ever informed by a price that had not printed. This is asserted by a regression test, and measured look-ahead is exactly `0.000e+00`.

I.0.3 Rebalance calendar

Code: `src/utils/backtesting.ts:379` — `periodKey()`

A rebalance fires on bar 0 (to establish the position) and whenever the **calendar bucket** changes:

- `weekly` → ISO week key (Thursday-anchored)
- `monthly` → `YYYY-M`
- `quarterly` → `YYYY-Q`
- `none` → bar 0 only

Why calendar buckets, not bar counts. On a sparse series (BRVM prints irregularly), "every 21 bars" is a variable number of months. Bucketing on the calendar makes "monthly" mean a month.

I.0.4 Transaction costs

Code: `src/utils/backtesting.ts:408` — `tradeCost()`

$$c = \sum_i \left| w_i^{\text{target}} - w_i^{\text{current}} \right| \times (f + s)$$

Charged on the **full notional traded** — no halving. Selling 5% of one holding to buy 5% of another moves 10% of the portfolio across the spread.

Example — opening trade. From flat $(0, 0, 0)$ to $(0.40, 0.40, 0.20)$ with $f + s = 0.0015$:

$$\text{notional} = 0.40 + 0.40 + 0.20 = 1.00, \qquad c = 1.00 \times 0.0015 = \mathbf{0.0015}$$

On $10,000 that is **15.00**, and the equity curve starts at **$9,985**, not $10,000.

Example — a monthly rebalance. From $(0.45, 0.35, 0.20)$ to $(0.40, 0.40, 0.20)$:

$$\text{notional} = 0.05 + 0.05 + 0.00 = 0.10, \qquad c = 0.10 \times 0.0015 = \mathbf{0.00015}$$

$1.50 on a $10,000 book. The reported **one-way turnover** for this event is $0.10/2 = 0.05$ (§I.2.23).

I.0.5 Equity recursion

Code: `src/utils/backtesting.ts:413` — `runBacktest()`

$$V_{t+1} = V_t \left(1 + \sum_i w_{i,t}\, r_{i,t} + c_t \cdot \frac{R_f}{P} \right)$$

where c_t is the cash weight (negative under leverage, in which case it *pays* the risk-free rate rather than earning it). At a rebalance, V_t is first reduced by the trade cost: $V_t \leftarrow V_t(1 - c)$.

Between rebalances positions **drift with prices** and are renormalized, which is what creates the tracking error the next rebalance must trade away:

$$w_{i,t+1} = \frac{w_{i,t}(1 + r_{i,t})}{\sum_j w_{j,t}(1 + r_{j,t}) + c_t(1 + R_f/P)}$$

I.1 — Tab: Results

Twelve cards, then the Equity Curve vs Benchmark chart. Listed in the exact on-screen order (left to right, top row then bottom row).

I.1.1 · TOTAL RETURN

Code: `src/utils/backtesting.ts:629`

$$\text{Total Return} = \frac{V_N}{V_0} - 1$$

Important: V_0 is the equity value **after the opening trade cost**, not the initial capital typed into the form.

Example (Dataset E). $V_0 = 10{,}000$, $V_{10} = 10{,}162$:

$$\frac{10{,}162}{10{,}000} - 1 = \mathbf{0.016200} \rightarrow \mathbf{1.62}\%$$

Card colouring: green when ≥ 0, red otherwise. Displayed to 2 dp.

I.1.2 · CAGR

Code: `src/utils/backtesting.ts:115` — `calculateCAGR()`

$$\text{CAGR} = \left(\frac{V_N}{V_0}\right)^{1/y} - 1, \qquad y = \frac{N_{\text{obs}} - 1}{P}$$

Example (Dataset E, $P = 252$).

$$y = \frac{11 - 1}{252} = 0.0396825 \text{ years}$$

$$\text{CAGR} = 1.0162^{\,1/0.0396825} - 1 = 1.0162^{\,25.2} - 1 = \mathbf{0.499255} \rightarrow \mathbf{49.93}\%$$

> **Read this as a warning, not a forecast.** 1.62% earned over ten trading days annualizes to +49.93%. The arithmetic is correct; the *inference* is not, because ten observations carry almost no information about a year. The engine reports it faithfully and leaves the judgement to you — which is why the header always discloses how many bars were simulated.

The exponent is guarded: $\max(y, 10^{-9})$, so a one-bar series cannot divide by zero.

I.1.3 · ALPHA

Code: `src/utils/backtesting.ts:176` — `calculateAlpha()`

Jensen's alpha, annualized, on **arithmetic** means:

$$\alpha = \bar{R}_p \cdot P - \left[R_f + \beta\left(\bar{R}_m \cdot P - R_f\right)\right]$$

Example (Datasets E vs B, $R_f = 0$, $\beta = 0.742645$ from §I.1.4).

$$\bar{R}_p \cdot P = 0.503085, \qquad \bar{R}_m \cdot P = 0.503895$$

$$\alpha = 0.503085 - [0 + 0.742645 \times 0.503895] = 0.503085 - 0.374213 = \mathbf{0.128872}$$

→ **+12.89%** annualized.

Alpha can be positive while the strategy loses money. In the screenshot in Appendix B, the return is −11.54% and alpha is +7.00%: the benchmark fell further than β predicted the strategy should have. Alpha measures *outperformance relative to beta-scaled benchmark risk*, not profit.

I.1.4 · BETA

Code: `src/utils/backtesting.ts:163` — `calculateBeta()`

$$\beta = \frac{\mathrm{Cov}(R_p, R_m)}{\mathrm{Var}(R_m)} = \frac{\sum_t (r_{p,t} - \bar{r_p})(r_{m,t} - \bar{r_m})}{\sum_t (r_{m,t} - \bar{r_m})^2}$$

The $1/(n-1)$ factors cancel, so the code sums raw cross-products directly.

Example (Datasets E vs B).

$$\sum (r_p - \bar{r_p})(r_m - \bar{r_m}) = 0.01036912, \qquad \sum (r_m - \bar{r_m})^2 = 0.01396241$$

$$\beta = \frac{0.01036912}{0.01396241} = \mathbf{0.742645}$$

Equivalently $\mathrm{Cov} = 0.00115212$ and $\mathrm{Var} = 0.00155138$ after dividing both by $n - 1 = 9$ — same ratio.

Card colouring quirk: the badge is driven by `raw = beta - 1`, so a beta below 1 (defensive) shows red and above 1 shows green. That is a *display* choice about market sensitivity, not a verdict on the number.

Returns 0 when $n < 2$ or $\mathrm{Var}(R_m) = 0$.

I.1.5 · SHARPE

Code: `src/utils/backtesting.ts:123` — `calculateSharpe()`

$$S = \frac{\bar{r_p} \cdot P - R_f}{s(r_p) \cdot \sqrt{P}}$$

Numerator and denominator are both annualized before dividing. Returns 0 when volatility is 0.

Example (Dataset E, $R_f = 0$, $P = 252$).

$$\text{numerator} = 0.001996 \times 252 = 0.503085$$

$$\text{denominator} = 0.029386 \times \sqrt{252} = 0.029386 \times 15.87451 = 0.466488$$

$$S = \frac{0.503085}{0.466488} = \mathbf{1.078453}$$

Sampling error. By Lo (2002), $\mathrm{SE}(S) \approx \sqrt{(1 + S^2/2)/T_{\text{years}}}$. With $S = 1.08$ over one year, $\mathrm{SE} \approx 1.24$ — the ratio is not distinguishable from zero. Over five years it falls to ≈ 0.55. Treat Sharpe on short windows as indicative.

I.1.6 · SORTINO

Code: `src/utils/backtesting.ts:128` — `calculateSortino()`

$$\text{Sortino} = \frac{\bar{r_p} \cdot P - R_f}{\sigma_d}, \qquad \sigma_d = \sqrt{\frac{1}{n}\sum_t \min(r_t, 0)^2}\ \sqrt{P}$$

Three things this gets right that a naive implementation gets wrong:

1. Divides by n (all periods), **not** by the count of negative periods. Filtering to the negative subset under-samples the denominator and inflates Sortino.
2. Deviations are measured from a **0 target**, not from the negative subset's own mean. Re-centring on the subset mean systematically understates downside risk.
3. When $\sigma_d = 0$ *and* the mean return is positive, it returns $+\infty$ — genuinely unbounded risk-adjusted performance — rather than the misleading 0 that `dd > 0 ? m/dd : 0` would give. The UI renders that as ∞, not the literal string "Infinity".

Example (Dataset E). Negative returns are −0.010000, −0.029994, −0.040035, −0.020049, −0.020058:

$$\sum \min(r, 0)^2 = 0.00340672$$

$$\sigma_d = \sqrt{\frac{0.00340672}{10}} \times \sqrt{252} = 0.018457 \times 15.87451 = 0.292995$$

$$\text{Sortino} = \frac{0.503085}{0.292995} = \mathbf{1.717011}$$

Note $\sigma_d = 0.2930 < \sigma = 0.4665$, so Sortino (1.717) exceeds Sharpe (1.078) — expected when the loss distribution is no worse than the gain distribution.

I.1.7 · CALMAR

Code: `src/utils/backtesting.ts:634`

$$\text{Calmar} = \frac{\text{CAGR}}{|\text{MDD}|}$$

Example (Dataset E).

$$\text{Calmar} = \frac{0.499255}{|-0.050562|} = \mathbf{9.874161}$$

Returns 0 when MDD is exactly 0 (a curve that never drew down — the ratio is undefined, not infinite, in the engine's convention).

Inherits CAGR's short-window distortion: the numerator is annualized, the denominator is not, so short backtests produce inflated Calmar figures.

I.1.8 · MAX DRAWDOWN

Code: `src/utils/backtesting.ts:143` — `calculateMaxDrawdown()`

$$\text{MDD} = \min_t \frac{V_t - \text{Peak}_t}{\text{Peak}_t}, \qquad \text{Peak}_t = \max_{u \le t} V_u$$

Peak is a **running** maximum, so drawdown is always measured from the high-water mark, never from the start.

Example (Dataset E). Full drawdown series:

t	V_t	**Peak**	**Drawdown**
0	10,000	10,000	0.0000
1	10,200	10,200	0.0000
2	10,098	10,200	−0.010000
3	10,502	10,502	0.0000
4	10,187	10,502	−0.029994
5	10,391	10,502	−0.010569
6	9,975	10,502	−0.050181
7	10,175	10,502	−0.031137
8	9,971	10,502	**−0.050562** ← MDD
9	10,370	10,502	−0.012569
10	10,162	10,502	−0.032375

$$\text{MDD} = \mathbf{-0.050562} \rightarrow \mathbf{-5.06}\backslash\%$$

The same pass returns the average of this column, used in §I.2.8.

I.1.9 · ANNUAL VOL.

Code: `src/utils/backtesting.ts:120` — `calculateVolatility()`

$$\sigma_p = s(r_p)\,\sqrt{P}$$

Square-root-of-time scaling, which assumes i.i.d. returns. Real returns cluster, so this understates risk at long horizons.

Example (Dataset E).

$$\sigma_p = 0.029386 \times \sqrt{252} = \mathbf{0.466488} \rightarrow \mathbf{46.65}\backslash\%$$

Card colouring quirk: driven by `raw = −annualVol`, so it is always red. Volatility is never "positive news" in this UI's colour language.

I.1.10 · VAR 95%

Code: `src/utils/backtesting.ts:155` — `calculateVaR()`

$$\mathrm{VaR}_{95} = Q_{0.05}(R_p)$$

Historical (non-parametric) VaR: the 5th-percentile **single-period** return, computed with the type-7 quantile of §0.7. Reported as a **negative** number — it is a return, not a loss magnitude.

Example (Dataset E). From §0.7: $\mathbf{-0.035517} \to \mathbf{-3.55}\%$.

Interpretation: on the worst 5% of days, this strategy lost at least 3.55%.

> Distinct from the Quant Lab's VaR (§II.3.1), which is **parametric** (Gaussian), denominated in **dollars**, and scaled to a multi-day horizon. Same name, different construction.

I.1.11 · CVAR 95%

Code: `src/utils/backtesting.ts:158` — `calculateCVaR()`

$$\mathrm{CVaR}_{95} = \mathbb{E}[R_p \mid R_p \le \mathrm{VaR}_{95}]$$

Expected shortfall: the mean of every return at or below the VaR threshold.

Example (Dataset E). Threshold −0.035517. Only one return qualifies (−0.040035):

$$\mathrm{CVaR}_{95} = \mathbf{-0.040035} \to \mathbf{-4.00}\%$$

CVaR $\le$ VaR always. When the tail is empty (possible when the quantile lands exactly on the minimum), it falls back to the VaR value rather than returning NaN.

> **Small-sample caveat.** With 10 observations the 5% tail contains one point, so CVaR *is* that point. Meaningful tail estimation needs hundreds of observations — with 5 years of daily data (~1,260 bars) the tail holds ~63 points, which is adequate.

I.1.12 · WIN RATE

Code: `src/utils/backtesting.ts:643`

$$\text{Win Rate} = \frac{|\{\text{trades} : \text{PnL} > 0\}|}{|\text{trades}|}$$

Counted over **trade legs**, not over days. A leg is one asset held from one rebalance to the next (§I.4).

Example. 20 legs, 9 with positive net PnL, 11 negative:

$$\text{Win Rate} = \frac{9}{20} = \mathbf{0.4500} \to \mathbf{45.00}\backslash\%$$

Zero trades → 0, not NaN. Badge colouring uses `raw = winRate − 0.5`, so the break-even point is 50%, not 0%.

I.1.13 · Equity Curve vs Benchmark (chart)

Code: `src/components/backtesting/EquityCurveChart.tsx`

Three series:

Series	Formula
Strategy	V_t from §I.0.5
Benchmark	$B_t = \frac{P^m_t}{P^m_0} \times V_{\text{capital}}$
Initial capital	Constant V_{capital} (dashed)

The benchmark is **rebased to the same starting capital** so the two curves are directly comparable. It is a pure price series — it pays no fees and no slippage, which is why a buy-and-hold strategy on the benchmark itself would still trail it slightly.

Example. Benchmark BTC at $P^m_0 = 45{,}000$, $P^m_t = 52{,}000$, capital \$10,000:

$$B_t = \frac{52{,}000}{45{,}000} \times 10{,}000 = \mathbf{11{,}555.56}$$

Down-sampled to ~200 points via `step = floor(dates.length / 200)` for rendering only; all metrics use the full series.

I.2 — Tab: Advanced metrics

A 24-row table, split into two columns. Listed here **one-to-one in table order**. Rows that also appear on the Results tab are cross-referenced rather than re-derived — but the formatting differs, so that is noted.

#	Row label	Formula	Reference
1	Total Return	$V_N/V_0 - 1$	§I.1.1
2	Annualized Return (CAGR)	$(V_N/V_0)^{1/y} - 1$	§I.1.2
3	Annualized Volatility	$s(r)\sqrt{P}$	§I.1.9
4	Sharpe Ratio	$(\bar{r}P - R_f)/(s\sqrt{P})$	§I.1.5
5	Sortino Ratio	$(\bar{r}P - R_f)/\sigma_d$	§I.1.6
6	Calmar Ratio	$\mathrm{CAGR}/\lvert\mathrm{MDD}\rvert$	§I.1.7
7	Max Drawdown	$\min_t(V_t - \mathrm{Peak}_t)/\mathrm{Peak}_t$	§I.1.8
8	Average Drawdown	**new** — §I.2.8	
9	Skewness	**new** — §I.2.9	
10	Kurtosis (excess)	**new** — §I.2.10	
11	VaR 95%	$Q_{0.05}(R)$	§I.1.10
12	CVaR 95%	$\mathbb{E}[R \mid R \le \mathrm{VaR}]$	§I.1.11
13	Beta vs Benchmark	$\mathrm{Cov}(R_p, R_m)/\mathrm{Var}(R_m)$	§I.1.4
14	Annualized Alpha	$\bar{R}_p P - [R_f + \beta(\bar{R}_m P - R_f)]$	§I.1.3
15	Tracking Error	**new** — §I.2.15	
16	Information Ratio	**new** — §I.2.16	
17	Win Rate	wins / trades	§I.1.12
18	Profit Factor	**new** — §I.2.18	
19	Average Win	**new** — §I.2.19	
20	Average Loss	**new** — §I.2.20	
21	Best Day	**new** — §I.2.21	
22	Worst Day	**new** — §I.2.22	
23	Turnover (one-way)	**new** — §I.2.23	
24	Fees Paid	**new** — §I.2.24	
(25)	Exposure	*omitted by design* — §I.2.25	

Formatting differences from the Results tab. Ratios here render at **3 dp** (Results uses 2 dp); non-finite values render as — and $+\infty$ as ∞; dollar figures use `$x.xx`.

I.2.1 – I.2.7 · Repeated from Results

Identical computation, identical value. See §I.1.1, §I.1.2, §I.1.9, §I.1.5, §I.1.6, §I.1.7, §I.1.8 respectively.

Using Dataset E throughout:

Row	Value (3 dp)
Total Return	1.62%
Annualized Return (CAGR)	49.93%
Annualized Volatility	46.65%
Sharpe Ratio	1.078
Sortino Ratio	1.717
Calmar Ratio	9.874
Max Drawdown	−5.06%

I.2.8 · Average Drawdown

Code: `src/utils/backtesting.ts:143` (the `avg` field of `calculateMaxDrawdown`)

$$\overline{\mathrm{DD}} = \frac{1}{N} \sum_{t} \frac{V_t - \mathrm{Peak}_t}{\mathrm{Peak}_t}$$

Averaged over **every** bar, including bars at a new high (which contribute exactly 0). It is therefore "average distance below the high-water mark", not "average depth of a drawdown episode".

Example (Dataset E). Summing the drawdown column of §I.1.8 and dividing by 11:

$$\overline{\mathrm{DD}} = \frac{-0.227387}{11} = \mathbf{-0.020672} \to \mathbf{-2.07\%}$$

Compare to MDD −5.06%: the strategy spent most of its life about 2% below its peak.

I.2.9 · Skewness

Code: `src/utils/backtesting.ts:197` — `calculateSkewness()`

Population (biased) third standardized moment:

$$g_1 = \frac{1}{n} \sum_t \left(\frac{r_t - \bar{r}}{s} \right)^3$$

Note s uses the **sample** $(n-1)$ convention while the outer average uses n — this is the `scipy.stats.skew(bias=True)` variant modified to use the sample standard deviation. Returns 0 for $n < 3$ or $s = 0$.

Example (Dataset E).

$$g_1 = \mathbf{-0.010638}$$

Essentially symmetric — as designed, since Dataset E's gains and losses mirror each other. Negative skew means the left tail is longer: rare large losses, frequent small gains.

I.2.10 · Kurtosis (excess)

Code: `src/utils/backtesting.ts:204` — `calculateKurtosis()`

$$g_2 = \frac{1}{n} \sum_t \left(\frac{r_t - \bar{r}}{s} \right)^4 - 3$$

The -3 makes it **excess** kurtosis, so a Gaussian scores 0. Returns 0 for $n < 4$ or $s = 0$.

Example (Dataset E).

$$g_2 = \mathbf{-1.772170}$$

Strongly *platykurtic* — Dataset E is a near-bimodal ±2%/±4% pattern with no genuine tail, which is the opposite of real markets. Real daily equity returns typically show excess kurtosis of +3 to +8; crypto often exceeds +10.

> **This number is the health warning on §II.3.** A large positive excess kurtosis is direct evidence that the Gaussian VaR in the Quant Lab **understates** tail risk for that portfolio.

I.2.11 – I.2.14 · Repeated from Results

Row	Formula	Reference	Dataset E value
VaR 95%	$Q_{0.05}(R)$	§I.1.10	−3.55%
CVaR 95%	$\mathbb{E}[R \mid R \le \text{VaR}]$	§I.1.11	−4.00%
Beta vs Benchmark	Cov / Var	§I.1.4	0.743
Annualized Alpha	Jensen's α	§I.1.3	12.89%

I.2.15 · Tracking Error

Code: `src/utils/backtesting.ts:182` — `calculateTrackingError()`

$$\text{TE} = s(r_{p,t} - r_{m,t})\sqrt{P}$$

Annualized standard deviation of the **active return** (the difference series), not of the portfolio itself.

Example (Datasets E vs B). Difference series:

```
-0.010000,  0.010000, -0.009923,  0.010013, -0.009953,
 0.009970, -0.009985,  0.009988, -0.010054,  0.009912
```

$$s(\text{diff}) = 0.010520, \qquad \text{TE} = 0.010520 \times 15.87451 = \mathbf{0.166995} \to \mathbf{16.70}\%$$

I.2.16 · Information Ratio

Code: `src/utils/backtesting.ts:187` — `calculateInformationRatio()`

$$\text{IR} = \frac{\overline{(r_p - r_m)} \cdot P}{\text{TE}}$$

Active return per unit of active risk. Recomputes TE internally rather than taking it as an argument, so the two rows are always consistent. Returns 0 when TE is 0.

Example (Datasets E vs B).

$$\overline{\text{diff}} = -0.0000032, \qquad \overline{\text{diff}} \times 252 = -0.000810$$

$$\text{IR} = \frac{-0.000810}{0.166995} = \mathbf{-0.004850}$$

Effectively zero: Dataset E tracks its benchmark almost exactly on average, while accumulating 16.70% of tracking error doing so — a textbook picture of activity without edge.

> **IR and alpha can disagree in sign** (here: $\alpha = +12.89\%$, IR $= -0.005$). They measure different things — alpha is β-adjusted, IR is not. With $\beta = 0.74$, the strategy took less market risk, which alpha credits and IR ignores.

I.2.17 · Win Rate

Same as §I.1.12. Rendered at 2 dp with a percent sign.

I.2.18 · Profit Factor

Code: `src/utils/backtesting.ts:648`

$$\text{PF} = \frac{\sum_{\text{wins}} \text{PnL}}{\left|\sum_{\text{losses}} \text{PnL}\right|}$$

Returns `null` when there are no losing trades — the denominator is zero, so the ratio is undefined, not large. The UI renders `null` as —.

> A previous implementation returned a magic `99` in that case, which displayed as a real-looking "99.000" and read as a genuine measurement.

Example. Gross wins $2,400$, $gross losses$1,600:

$$\text{PF} = \frac{2400}{1600} = \mathbf{1.500}$$

PF > 1 means winners out-earn losers in aggregate. It is independent of win rate — a 30% win rate with large winners can beat a 70% win rate with small ones.

I.2.19 · Average Win

Code: `src/utils/backtesting.ts:688`

$$\overline{W} = \frac{\sum_{\text{wins}} \text{PnL}}{|\{\text{wins}\}|}$$

Example. $2,400 over 9 winners \rightarrow$ **266.67**. Zero winners → $0.00.

I.2.20 · Average Loss

Code: `src/utils/backtesting.ts:689`

$$\overline{L} = -\frac{|\sum_{\text{losses}} \text{PnL}|}{|\{\text{losses}\}|}$$

Reported as a **negative** dollar figure.

Example. $1,600 over 11 losers \rightarrow$ **−145.45**.

Cross-check against §I.2.18: $\text{PF} = \dfrac{9 \times 266.67}{11 \times 145.45} = 1.50$. ✓

I.2.21 · Best Day

Code: `src/utils/backtesting.ts:690`

$$\max_t r_{p,t}$$

Best single **period** return (a period is a bar, so "day" is literal for daily data).

Example (Dataset E). $\mathbf{0.040016 \rightarrow 4.00\%}$.

I.2.22 · Worst Day

Code: `src/utils/backtesting.ts:691`

$$\min_t r_{p,t}$$

Example (Dataset E). $\mathbf{-0.040035 \rightarrow -4.00\%}$.

Note Worst Day (−4.00%) is *worse* than VaR 95% (−3.55%) — necessarily so: VaR is the 5th percentile, Worst Day the 0th.

I.2.23 · Turnover (one-way)

Code: `src/utils/backtesting.ts:472`

$$\text{Turnover} = \sum_{\text{rebalances}} \frac{1}{2} \sum_{i} \left| w_i^{\text{new}} - w_i^{\text{old}} \right|$$

The $\frac{1}{2}$ makes it **one-way** — the conventional reporting basis, where "100% turnover" means the whole book was replaced once. Note this is a *cumulative* sum over the whole backtest, not an annual rate.

> **This differs from the cost basis on purpose.** §I.0.4 charges the *two-way* notional (no halving), because both the sale and the purchase cross the spread. Reporting one-way turnover while billing two-way notional is the industry convention, and confusing them is what caused costs to be understated by ~2× in an earlier version of this engine.

Example. Opening trade (0.5) plus 11 monthly rebalances averaging 0.05 each:

$$\text{Turnover} = 0.5 + 11 \times 0.05 = \mathbf{1.050}$$

Rendered as a bare number (1.050), meaning the book turned over 1.05 times.

I.2.24 · Fees Paid

Code: `src/utils/backtesting.ts:473`

$$\text{Fees} = \sum_{\text{rebalances}} c_t \cdot V_t$$

Actual dollars removed from equity, accumulated at each rebalance using the equity value **at that moment** — so a rebalance late in a profitable run costs more in absolute terms than the same weight change early on.

Example. Opening trade $15.00 (\S I.0.4) plus 11 rebalances at \approx$1.50 on ≈ $10,000:

$$\text{Fees} \approx 15.00 + 11 \times 1.50 = \mathbf{\$31.50}$$

Cross-check: this should equal $V_{\text{capital}} - V_0$ plus the sum of later charges. In Appendix B's real run, $10{,}000 - 9{,}985 = \$15$ is exactly the opening charge.

I.2.25 · Exposure — omitted

Code: `src/utils/backtesting.ts:696`

Set to `null` and **not rendered**. The row appears only if a future engine version supplies a real value.

Most strategies here are fully invested at all times, so exposure would be 1 by construction rather than by measurement — a constant "100.00%" that looks computed and is not. Two strategies *do* have a genuine cash leg (`sma-crossover` when nothing is bullish, `vol-targeting` always), so a real exposure figure is computable; it simply is not wired to this row yet.

I.3 — Tab: Curves

Six charts. Each restates a metric from §I.1/§I.2 as a time series.

I.3.1 · Equity Curve

Same three series as §I.1.13, rendered again in the two-column grid.

I.3.2 · Drawdown

Code: `src/components/backtesting/DrawdownChart.tsx`

$$\mathrm{DD}_t = \frac{V_t - \mathrm{Peak}_t}{\mathrm{Peak}_t} \times 100$$

The full series whose minimum is §I.1.8 and whose mean is §I.2.8. Plotted as a filled area below zero.

Example (Dataset E). The column tabulated in §I.1.8, ×100: `0, 0, −1.00, 0, −3.00, −1.06, −5.02, −3.11, −5.06, −1.26, −3.24` (%).

I.3.3 · Rolling Volatility

Code: `src/utils/backtesting.ts:525`; window constant `ROLLING_VOL_WINDOW = 30`

$$\sigma_t = s(r_{t-29}, \ldots, r_t)\sqrt{P}$$

Emits `null` until the 30-period window is genuinely full, and the chart drops those points. A partial window produces a value that ramps up from 0 (the standard deviation of a one-element slice *is* 0), which renders as a collapse in volatility rather than as missing data.

Example. With 250 aligned bars, the line begins at bar 30 and carries 220 points. The legend reads the window size from the exported constant, so it cannot drift out of step with the engine.

I.3.4 · Rolling Sharpe

Code: `src/utils/backtesting.ts:530`; window constant `ROLLING_SHARPE_WINDOW = 60`

$$S_t = \frac{\overline{r_{t-59..t}} \cdot P - R_f}{s(r_{t-59..t}) \cdot \sqrt{P}}$$

Same null-until-full policy. A 60-period window is the shortest that gives Sharpe any stability at all — see the standard-error note in §I.1.5.

A zero reference line is drawn, so sign changes are readable at a glance.

I.3.5 · Monthly Returns Heatmap

Code: `src/utils/backtesting.ts:537`; `src/components/backtesting/MonthlyReturnsHeatmap.tsx`

Monthly return — geometric, measured against the **previous month's closing equity**:

$$R_{y,m} = \frac{V_{\text{last bar of month } m}}{V_{\text{last bar of month } m-1}} - 1$$

The first month of the backtest has no predecessor, so its base is the opening equity V_0.

> **Not the month's own first bar.** The move from one month's last bar to the next month's first bar is a real return; basing each month on its own first bar would assign it to no month at all, and the YTD column below would then fail to reconcile with total return — by more every year, since the shortfall compounds along the row.

YTD column — compounded, not summed:

$$\text{YTD}_y = \prod_{m} (1 + R_{y,m}) - 1$$

Example. Jan +3.1%, Feb −1.8%, Mar +4.2%, Apr +0.7%:

$$1.031 \times 0.982 \times 1.042 \times 1.007 - 1 = \mathbf{0.062349} \rightarrow \mathbf{+6.23\%}$$

Simple addition would give +6.20% — the 0.03 pp gap is the compounding, and it widens with volatility.

Cell shading.

$$\text{intensity} = \min\left(1, \frac{|R_{y,m}|}{\max_{y,m} |R_{y,m}|}\right), \qquad \text{alpha} = 0.15 + 0.55 \times \text{intensity}$$

Green for ≥ 0, red otherwise; months with no data render as **—** with a transparent background, never as 0%.

I.3.6 · Allocation Over Time

Code: `src/routes/_authenticated/backtesting.tsx` (stacked `AreaChart`, `stackOffset="expand"`)

Plots $w_{i,t}$ at each **rebalance event**, normalized to 100% of the stack height.

$$\text{stack}_i(t) = \frac{w_{i,t}}{\sum_j w_{j,t}}$$

Example. A max-Sharpe strategy on Dataset R (§Appendix A) rebalancing monthly would show bands at roughly 3% / 4% / 93% — heavily concentrated in the low-volatility asset — and those bands would move each month as the trailing estimation window rolls forward.

Under Buy & Hold with no rebalancing there is a single point, so the chart is a flat stack: the weights genuinely never change.

> The `stackOffset="expand"` normalization means a **vol-targeting** strategy's cash leg is invisible here — the risky weights are rescaled to fill the chart. Read cash from `allocationOverTime[].cash` rather than from this chart.

I.4 — Tab: Trades

I.4.0 · How a trade leg is constructed

Code: `src/utils/backtesting.ts:575`

Trades are derived from the **actual rebalance events the engine recorded**, not from a synthetic fixed-interval schedule. One leg = one asset, held from rebalance e to rebalance $e + 1$, at the weight rebalance e assigned.

$$q = \frac{V_{t_0} \cdot w_i}{P_{i,t_0}}$$

$$\text{fees} = (P_{i,t_0} + P_{i,t_1}) \cdot q \cdot (f + s)$$

$$\text{PnL} = (P_{i,t_1} - P_{i,t_0}) \cdot q - \text{fees}$$

$$\text{PnL}\% = \frac{\text{PnL}}{P_{i,t_0} \cdot q}$$

Positions below **0.5% weight** are skipped — too small to represent a real trade. `side` is `buy` when $w_i \geq w_i^{\text{prev}}$ and `sell` otherwise (no shorting exists in this engine; every leg is long).

Worked example.

Input	Value
Equity at t_0	$10,000
Weight	0.40
Entry price	$180.00
Exit price	$195.00
$f + s$	0.0015

$$q = \frac{10{,}000 \times 0.40}{180.00} = 22.2222 \text{ units}$$

$$\text{fees} = (180 + 195) \times 22.2222 \times 0.0015 = 375 \times 22.2222 \times 0.0015 = \mathbf{\$12.50}$$

$$\text{PnL} = (195 - 180) \times 22.2222 - 12.50 = 333.33 - 12.50 = \mathbf{\$320.83}$$

$$\text{PnL}\% = \frac{320.83}{180 \times 22.2222} = \frac{320.83}{4000} = \mathbf{8.02\%}$$

Status: `win` (PnL > 0), `loss` (< 0), `flat` (exactly 0).

> Note the fee model here charges **both** legs of the round trip at entry and exit prices — a different accounting basis from §I.0.4, which charges portfolio-level weight changes. The per-trade figure is an attribution view; §I.2.24 is the authoritative total.

I.4.1 – I.4.7 · Summary tiles

Code: `src/components/backtesting/TradeAnalysisTable.tsx`

#	Tile	Formula	Example
1	Number of trades	$\|\|\text{trades}\|\|$	20
2	Winners	$\|\|\{\text{PnL} > 0\}\|\|$	9
3	Losers	$\|\|\{\text{PnL} < 0\}\|\|$	11
4	Avg win	$\frac{\sum_{\text{wins}} \text{PnL}}{\max(\|\|\text{wins}\|\|, 1)}$	$2400/9 = \$266.67$
5	Avg loss	$\frac{\|\sum_{\text{losses}} \text{PnL}\|}{\max(\|\|\text{losses}\|\|, 1)}$	$1600/11 = \$145.45$
6	Profit factor	$\frac{\sum_{\text{wins}}}{\|\sum_{\text{losses}}\|}$	$2400/1600 = 1.50$
7	Avg duration	$\frac{1}{\|\|\text{trades}\|\|} \sum_k \text{durationDays}_k$	30

Two differences from §I.2, worth knowing:

1. **Profit factor with no losers.** This tile computes it locally and yields **0.00** when there are no losing trades; the Advanced metrics row (§I.2.18) yields `null` → —. The Advanced row is the correct one.
2. **Avg win / Avg loss** use $\max(\text{count}, 1)$ in the denominator, so with zero winners the tile shows $0.00 rather than NaN. §I.2.19/§I.2.20 branch on the count instead. Same answer whenever the count is non-zero.

Duration:

$$\text{durationDays} = \text{round}\left(\frac{t_1 - t_0}{86{,}400{,}000} \right)$$

Calendar days between entry and exit dates — so a monthly rebalance shows ~30, not the ~21 trading bars that actually elapsed.

I.4.8 · Trade table columns

Column	Source
Entry / Exit	`dates[t0]`, `dates[t1]`
Asset	Symbol
Side	`buy` if $w \geq w^{\text{prev}}$, else `sell`
Qty	q (§I.4.0)
Entry price / Exit price	P_{i,t_0}, P_{i,t_1}
PnL	§I.4.0
PnL %	§I.4.0
Duration	Calendar days
Fees	§I.4.0
Status	win / loss / flat

Paginated 20 rows at a time.

I.5 — Tab: Models

Eighteen formula cards rendered in KaTeX, plus a **View Python code** button. Each card states the formula the engine *actually evaluates* — the Markowitz notation is accurate rather than aspirational, and was deliberately withdrawn while the engine ran diagonal approximations.

Cards 1–11 restate metrics already covered above; cards 12–18 are the allocation machinery that appears nowhere else in the UI.

I.5.1 · Simple return

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

See §0.4. Example: **+2.00%**.

I.5.2 · Log return

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

See §0.5. Example: **0.019803**.

I.5.3 · Portfolio return

Code: `src/utils/backtesting.ts:105` — `calculatePortfolioReturns()`

$$R_{p,t} = \sum_i w_i \, R_{i,t}$$

Example (Dataset R, $w = (0.40, 0.40, 0.20)$), first three bars:

$$R_{p,1} = 0.40(0.010) + 0.40(0.020) + 0.20(0.003) = 0.004 + 0.008 + 0.0006 = \mathbf{0.012600}$$

$$R_{p,2} = 0.40(-0.020) + 0.40(-0.030) + 0.20(0.001) = \mathbf{-0.019800}$$

$$R_{p,3} = 0.40(0.030) + 0.40(0.050) + 0.20(-0.002) = \mathbf{0.031600}$$

I.5.4 · Portfolio volatility

$$\sigma_p = \mathrm{stdev}(R_p)\sqrt{P}$$

See §I.1.9. Example: **46.65%**.

> The optimizers in §I.5.12–§I.5.15 use the *quadratic form* $\sqrt{w^{\mathsf{T}}\Sigma w}$ instead, which accounts for correlation. These agree only when computed on the same data; the card here describes the realized-series version.

I.5.5 · Sharpe ratio

$$S = \frac{\mathbb{E}[R_p] - R_f}{\sigma_p}$$

See §I.1.5. Example: **1.078**.

I.5.6 · Sortino ratio

$$\text{Sortino} = \frac{\mathbb{E}[R_p] - R_f}{\sigma_d}, \quad \sigma_d = \sqrt{\frac{1}{n}\sum_{t} \min(r_t, 0)^2}\sqrt{P}$$

See §I.1.6. Example: **1.717**.

I.5.7 · Max drawdown

$$\text{MDD} = \min_{t} \left(\frac{V_t - \text{Peak}_t}{\text{Peak}_t} \right)$$

See §I.1.8. Example: **−5.06%**.

I.5.8 · VaR 95%

$$\text{VaR}_{95} = \text{Quantile}_{5\%}(R)$$

See §I.1.10. Example: **−3.55%**.

I.5.9 · CVaR

$$\text{CVaR} = \mathbb{E}[R \mid R \leq \text{VaR}]$$

See §I.1.11. Example: **−4.00%**.

I.5.10 · Beta

$$\beta = \frac{\text{Cov}(R_p, R_m)}{\text{Var}(R_m)}$$

See §I.1.4. Example: **0.743**.

I.5.11 · Alpha

$$\alpha = R_p - [R_f + \beta(R_m - R_f)]$$

See §I.1.3. Example: **+12.89%**.

I.5.12 · Minimum variance

Code: `src/calculations/optimize.ts:61` — `minVariance()`

$$\min_{w} \; w^{\mathsf{T}}\Sigma\, w \quad \text{s.t.} \;\; \sum_i w_i = 1,\; w_i \geq 0$$

Algorithm — projected gradient descent. The objective is convex and the feasible set is convex, so this converges to the **global** optimum.

$$\nabla = 2\Sigma w, \qquad w \leftarrow \Pi_{\Delta}(w - \eta\nabla), \qquad \eta = \frac{1}{2\,\lambda_{\max}^{\text{bound}}(\Sigma)}$$

500 iterations from an equal-weight start. Deterministic — no randomness, so identical inputs give identical weights.

Step size comes from a **Gershgorin circle bound** on the largest eigenvalue (`covariance.ts:96` — `spectralBound()`), which guarantees stability without an eigendecomposition:

$$\lambda_{\max} \leq \max_i \sum_j |\Sigma_{ij}|$$

Simplex projection (`optimize.ts:29` — `projectOntoSimplex()`), Duchi et al. (2008): sort descending, find $\rho = \max\{j : u_j - \theta_j > 0\}$ where $\theta_j = \left(\sum_{k\leq j} u_k - 1\right)/j$, then $w_i = \max(0, v_i - \theta)$.

This enforces long-only + fully-invested at **every step**, rather than clipping negatives afterwards (which breaks the sum) or renormalizing (which reintroduces them proportionally).

Example. $\Pi_{\Delta}([0.7, 0.5, -0.2]) = [\mathbf{0.60,\ 0.40,\ 0.00}]$ — sums to 1, non-negative, and is the nearest such point in Euclidean distance.

Example (Dataset R, shrunk Σ^* from §I.5.16).

$$\lambda_{\max}^{\text{bound}} = 0.240083 \;\Rightarrow\; \eta = \frac{1}{2 \times 0.240083} = 2.082611$$

$$w^{\text{minvar}} = [\mathbf{0.0274,\ 0.0137,\ 0.9588}]$$

Resulting volatility **2.76%**, versus **19.64%** equal-weighted — a 7× reduction, achieved by concentrating in the low-volatility asset C ($\sigma_C = 3.00\%$).

I.5.13 · Maximum Sharpe (tangency)

Code: `src/calculations/optimize.ts:104` — `maxSharpe()`

$$\max_{w} \ \frac{w^{\mathsf{T}}\mu - R_f}{\sqrt{w^{\mathsf{T}}\Sigma\, w}} \quad \text{s.t.} \ \ \sum_i w_i = 1,\ w_i \geq 0$$

Gradient:

$$\nabla = \frac{\mu}{\sigma} - \frac{(w^{\mathsf{T}}\mu - R_f)\,\Sigma w}{\sigma^3}, \qquad \sigma = \sqrt{w^{\mathsf{T}}\Sigma w}$$

Step schedule — normalized direction with $1/\sqrt{k}$ decay:

$$w \leftarrow \Pi_\Delta\!\left(w + \frac{\eta_k}{\|\nabla\|}\nabla\right), \qquad \eta_k = \frac{\sqrt{2}}{\sqrt{k+1}}$$

$\sqrt{2}$ is the **diameter of the simplex** (the distance between two distinct vertices) — no step needs to exceed it. The routine tracks the best Sharpe seen across all 500 iterations and returns that, so it can never return a point worse than its starting equal-weight portfolio.

> **Why the step schedule is different from §I.5.12.** The fixed step $1/(2\lambda_{\max})$ is derived from the smoothness of the **variance** objective, whose curvature is bounded by $\lambda_{\max}(\Sigma)$. The Sharpe objective is a *ratio* and scales as $1/\sigma$, so the same step is wildly wrong for it. Measured on a typical 6-asset annualized covariance: $\lambda_{\max}^{\text{bound}} = 0.163$ gives $\eta = 3.06$, and with $\|\nabla\| = 1.65$ each iteration displaced the weights by 5.04 — about **3.6× the entire feasible set's diameter**. Every step overshot, the projection snapped to a vertex, and the routine returned a single-asset corner in 28 of 60 randomly generated problems, falling short of the true optimum by up to 21% of the attainable Sharpe. Verified against SLSQP with 25 random restarts over the same 60 problems: **49/49 reach the optimum** under the current schedule, where the previous step reached 24/49, with zero measured shortfall.

Fallback. When no asset offers a positive excess return ($\mu_i - R_f \leq 0\ \forall i$), the Sharpe objective has no meaningful maximum, so it delegates to §I.5.12.

Non-concavity. Unlike min-variance, this objective is not concave in general, so the result is a **local** optimum. It is deterministic, which matters more here — a backtest returning different weights on identical inputs would be untestable.

Example (Dataset R, $R_f = 0$, shrunk Σ^*).

$$\mu = [1.0080,\ 1.7640,\ 0.3276]\ \text{(annualized)}$$

$$w^{\text{maxSharpe}} = [\mathbf{0.0326,\ 0.0398,\ 0.9276}]$$

Portfolio Sharpe **13.47**, versus **5.26** equal-weighted. (The absolute level is an artifact of annualizing 10 synthetic observations; the *ratio* between the two is the meaningful part.)

Note the optimizer picks asset C at 93% despite it having by far the lowest return (32.76% vs 176.40%) — because $\sigma_C = 3.00\%$ against $\sigma_B = 43.67\%$, and Sharpe rewards the ratio, not the numerator.

I.5.14 · Risk parity (equal risk contribution)

Code: `src/calculations/optimize.ts:153` — `riskParity()`

$$RC_i = \frac{w_i\,(\Sigma w)_i}{w^{\mathsf{T}}\Sigma w} = \frac{1}{n} \quad \forall i$$

Algorithm — multiplicative updates from an inverse-volatility start:

$$w_i \leftarrow w_i \sqrt{\frac{1/n}{RC_i}}, \qquad \text{then renormalize } w \leftarrow \frac{w}{\sum_j w_j}$$

The square root damps the step; a raw ratio oscillates on strongly-correlated inputs. 500 iterations.

> **Inverse-volatility weighting is *not* risk parity** — it is the solution only in the special case where every pairwise correlation is identical, which is exactly the assumption a risk-parity portfolio exists to avoid making. It is used here as a *starting point*, so the iteration begins close to the answer and only has to correct for correlation structure.

Example (Dataset R, shrunk Σ^*).

$$w^{\text{ERC}} = [\mathbf{0.0740,\ 0.0463,\ 0.8797}]$$

Verification — the risk contributions:

$$RC = [0.333333,\ 0.333333,\ 0.333333], \qquad \sum RC = 1.000000\ \checkmark$$

Exactly $1/3$ each, to six decimals. Resulting portfolio volatility **3.61%**.

Contrast — what equal *weights* do to risk. At $w = (0.40, 0.40, 0.20)$ on the same matrix:

$$RC = [0.3384,\ 0.6684,\ -0.0068]$$

Asset B carries **67%** of the portfolio's risk on 40% of its capital, and asset C's contribution is *negative* (it hedges, via its −0.61 / −0.62 correlations). This gap between capital weight and risk weight is precisely what risk parity exists to close.

I.5.15 · Volatility targeting

Code: `src/calculations/optimize.ts:206` — `volatilityTarget()`

$$k = \min\left(L_{\max},\ \frac{\sigma_{\text{target}}}{\sqrt{w_{\text{ERC}}^{\mathsf{T}}\,\Sigma\,w_{\text{ERC}}}}\right), \qquad w_t = k\,w_{\text{ERC}}, \qquad c_t = 1 - k$$

Levers the risk-parity portfolio of §I.5.14 up or down toward a target volatility. **The residual is held as cash**, earning R_f — and may go negative, which is borrowing.

> **This strategy was previously inert.** The old implementation scaled the weights and then renormalized them to sum to 1, which cancelled the scaling exactly, making it algebraically identical to plain risk parity. The strategy is meaningless without somewhere for the difference to go. Realized volatility is also now the correct quadratic form $\sqrt{w^{\mathsf{T}}\Sigma w}$; the old code used $\sqrt{\sum_i (w_i\sigma_i)^2}$, which assumes zero correlation between every pair.

Example A — leverage cap binds. Dataset R, $\sigma_{\text{target}} = 12\%$, $L_{\max} = 1.5$:

$$\sigma_{\text{ERC}} = 3.61\%, \qquad \frac{0.12}{0.0361} = 3.33 \ \longrightarrow\ k = \min(1.5,\ 3.33) = \mathbf{1.50}$$

$$w_t = [\mathbf{0.1110,\ 0.0694,\ 1.3196}], \qquad c_t = 1 - 1.5 = \mathbf{-0.50}$$

Weights sum to **1.50**, not 1 — that is the point. Cash is −50%: the portfolio has borrowed half its own value. Realized volatility reaches only $1.5 \times 3.61\% = 5.41\%$, well short of the 12% target, because the cap binds first.

Example B — target binds (de-levering). Same portfolio, $\sigma_{\text{target}} = 2\%$:

$$k = \min\left(1.5,\ \frac{0.02}{0.036076}\right) = \mathbf{0.5544}$$

$$w_t = [\mathbf{0.0410,\ 0.0257,\ 0.4877}], \qquad c_t = \mathbf{0.4456}$$

Realized volatility lands on **exactly 2.00%** — the target is hit, with 44.56% parked in cash earning R_f.

I.5.16 · Covariance shrinkage

Code: `src/calculations/covariance.ts:13` (`sampleCovariance`), `:53` (`shrinkCovariance`), `:66` (`suggestedShrinkage`)

Sample covariance, annualized:

$$\Sigma_{ij} = \frac{P}{T-1} \sum_{t=1}^{T} \left(r_{i,t} - \bar{r}_i\right)\left(r_{j,t} - \bar{r}_j\right)$$

The $(T-1)$ convention matches §0.6, so the diagonal equals the squared volatilities reported everywhere else.

Shrinkage toward the diagonal:

$$\Sigma^{*} = (1-\delta)\,\Sigma + \delta\ \mathrm{diag}(\Sigma)$$

which in code is simply: leave the diagonal alone, multiply every off-diagonal by $(1-\delta)$.

Suggested intensity:

$$\delta = \mathrm{clip}\left(\frac{n(n+1)/2}{T},\ 0.05,\ 0.90\right)$$

the ratio of parameters being estimated to observations available.

> **Not optional at these sample sizes.** A sample covariance from T observations of n assets is **singular** whenever $T \le n$, and badly conditioned well before that. An optimizer descending on an ill-conditioned matrix produces extreme, unstable weights that look precise and are not. Shrinking toward the diagonal keeps the volatilities (estimated far more reliably than the correlations) and pulls the off-diagonals toward zero in proportion to how little data supports them. Ledoit–Wolf optimal intensity is the natural refinement; a documented fixed rule is preferred here because it is trivially reproducible in the Python reference.

Example — intensity scaling:

n	T	$n(n+1)/2$	**Raw ratio**	δ
3	252	6	0.024	**0.05** (floor binds)
3	10	6	0.600	**0.60**
5	30	15	0.500	**0.50**
8	60	36	0.600	**0.60**
25	252	325	1.290	**0.90** (cap binds)

Example — Dataset R ($n = 3$, $T = 10$, $\delta = 0.60$).

Sample covariance (annualized):

$$\Sigma = \begin{pmatrix} 0.073920 & 0.115360 & -0.004956 \\ 0.115360 & 0.190680 & -0.008148 \\ -0.004956 & -0.008148 & 0.000899 \end{pmatrix}$$

After shrinkage — off-diagonals × 0.40:

$$\Sigma^{*} = \begin{pmatrix} 0.073920 & 0.046144 & -0.001982 \\ 0.046144 & 0.190680 & -0.003259 \\ -0.001982 & -0.003259 & 0.000899 \end{pmatrix}$$

Implied volatilities (unchanged by shrinkage — that is the design):

$$\sigma = \left[\sqrt{0.073920},\ \sqrt{0.190680},\ \sqrt{0.000899}\right] = [\mathbf{27.19\%},\ \mathbf{43.67\%},\ \mathbf{3.00\%}]$$

Implied correlation matrix (`covariance.ts:106` — `covToCorr()`, from the *unshrunk* matrix):

$$\rho = \begin{pmatrix} 1.0000 & 0.9717 & -0.6080 \\ 0.9717 & 1.0000 & -0.6224 \\ -0.6080 & -0.6224 & 1.0000 \end{pmatrix}$$

A 0.97 correlation from 10 observations is exactly the kind of estimate that should be disbelieved — hence $\delta = 0.60$, which pulls the effective correlation down to $0.40 \times 0.9717 = 0.389$.

Where the UI reports this. The header line under the Run button: *"estimation window: L bars, R rebalances, δ% shrinkage."*

I.5.17 · Transaction costs

$$c = \sum_i \left| w_i^{\text{target}} - w_i^{\text{current}} \right| \times (f + s)$$

See §I.0.4. Examples: opening trade **0.0015** ($15 on 10,000); typical monthly rebalance **0.00015** ($1.50).

I.5.18 · Estimation window

$$\Sigma_t,\ \mu_t = f(r_{t-L},\ \ldots,\ r_{t-1})$$

The look-ahead guarantee, stated as a formula. Note the window ends at $t-1$: **the current bar is excluded**. See §I.0.1 and §I.0.2.

μ_t is the annualized sample mean of the window, $\mu_i = \bar{r}_i \cdot P$; Σ_t is §I.5.16 applied to the same window.

I.5.19 · Strategies that use no optimizer

Four of the nine strategies bypass §I.5.12–§I.5.15 entirely.

Code: `src/utils/backtesting.ts:225` — `strategyWeights()`

Buy & Hold — user weights, honoured as given:

$$w = w^{\text{user}}, \qquad c = 1 - \sum_i w_i^{\text{user}}$$

An allocation summing to less than 1 is **held as cash**, not silently scaled up. Over-allocation ($\sum > 1$) *is* normalized.

Equal Weight: $w_i = 1/n$.

Momentum — positive-drift tilt over the lookback window only:

$$w_i = \frac{\max(0, \mu_i)}{\sum_j \max(0, \mu_j)}$$

Falls back to equal-weight when no asset has positive drift.

Example (Dataset R, $\mu = [1.008, 1.764, 0.3276]$): sum = 3.0996, so $w = [0.3252,\ 0.5691,\ 0.1057]$.

Inverse-Drift Weighting (union member `mean-reversion`, kept only to avoid breaking persisted configs):

$$w_i = \frac{1/(|\mu_i| + 0.05)}{\sum_j 1/(|\mu_j| + 0.05)}$$

> **This is not mean reversion.** No deviation from a moving average or long-run mean is measured anywhere. It tilts *away* from assets with large absolute drift in either direction. The UI label is "Inverse-Drift Weighting", which is accurate.

Example (Dataset R): $1/(1.008 + 0.05) = 0.9452$, $1/(1.764 + 0.05) = 0.5513$, $1/(0.3276 + 0.05) = 2.6484$; sum = 4.1449 → $w = [0.2280,\ 0.1330,\ 0.6390]$.

SMA Crossover — trend filter evaluated at the allocation date on the trailing window only:

$$\text{bullish}_i = \left[\overline{P_{i,\,t-19..t}} > \overline{P_{i,\,t-49..t}}\right], \qquad w_i = \frac{\text{bullish}_i}{\sum_j \text{bullish}_j}$$

Assets with fewer than 50 bars in the window default to bullish. **When nothing is bullish it holds 100% cash** rather than forcing an equal-weight long book.

Example: 20-bar SMA = $184.20, 50 - barSMA =$179.60 → bullish. If 2 of 3 assets are bullish, each bullish asset gets 0.50 and the third gets 0.

I.5.20 · Python code export

Code: `src/components/backtesting/PythonCodeModal.tsx`; reference at `quant_reference/polefinance_quant.py`

The **View Python code** button opens a NumPy/SciPy-free reference implementation of every formula on this page.

> **One deliberate deviation from idiomatic Python.** The reference defines its own `_sum()` naive left-fold and uses it in place of the builtin `sum()` on 34 lines. CPython 3.12+ ships **Neumaier compensated summation** in `sum()`, which is *more* accurate than JavaScript's `Array.reduce` — and that difference is enough to break bit-for-bit parity between the two implementations. Matching the JS engine exactly is the goal, so the reference reproduces the less accurate summation on purpose.

Part II — Quant Lab

Three tabs. **They do not share an input source**, which is the single most common point of confusion:

Tab	Inputs from
Monte Carlo	The **asset selector on that tab**; drift/vol calibrated from each asset's own price history
Stress Tests	Your **actual portfolio holdings** (or a disclosed demo portfolio if you hold nothing)
VaR / CVaR	**Manual number entry**, with a "Use my portfolio" button to prefill

> This is why the Stress Tests tab has no asset selector: it stresses what you own, not what you picked. If you hold nothing, it falls back to a fixed demo portfolio (BTC $4,000/ETH$2,000 / AAPL $1,500/NVDA$500 / cash $2,000) and says so in a banner.

II.1 — Tab: Monte Carlo

II.1.0 · Calibration from real prices

Code: `src/utils/backtesting.ts:829` — `realizedDriftVol()`

Each asset's drift and volatility are estimated from **its own** price history, *not* cross-intersected with the other selected assets — because correlation on this tab is the explicit slider, not something derived from real co-movement.

$$\mu_i = \bar{r}_i \cdot P_i, \qquad \sigma_i = s(r_i)\,\sqrt{P_i}$$

Each asset uses its own inferred P_i (§0.3), so BTC annualizes on ~365 and AAPL on ~252 — as they should.

II.1.1 · Portfolio drift and volatility

Code: `src/utils/quantLab.ts:76` — `runMonteCarlo()`

Weights are normalized, then drift is a plain weighted average and variance uses a **constant-correlation** matrix:

$$W_i = \frac{w_i}{\sum_j w_j}, \qquad \mu_p = \sum_i W_i \mu_i$$

$$\sigma_p^2 = \sum_i \sum_j W_i W_j \sigma_i \sigma_j \, c_{ij}, \qquad c_{ij} = \begin{cases} 1 & i = j \\ \rho & i \neq j \end{cases}$$

Example. BTC 40% (μ 35%, σ 60%), ETH 30% (μ 30%, σ 75%), AAPL 30% (μ 12%, σ 28%), $\rho = 0.30$:

$$\mu_p = 0.40(0.35) + 0.30(0.30) + 0.30(0.12) = 0.14 + 0.09 + 0.036 = \mathbf{0.266000}$$

$$\sigma_p^2 = \mathbf{0.171117} \;\Rightarrow\; \sigma_p = \sqrt{0.171117} = \mathbf{0.413663}$$

(For comparison, the naive uncorrelated figure $\sqrt{\sum (W_i \sigma_i)^2} = 0.339531$ — the correlation term adds **7.41 pp** of volatility. Ignoring it would understate this portfolio's risk by **17.9%**.)

II.1.2 · PSD guard on correlation

Code: `src/utils/quantLab.ts:22` — `minAttainableCorrelation()`

An equicorrelation matrix has eigenvalues $(1 - \rho)$ with multiplicity $n - 1$ and $(1 + (n-1)\rho)$ once. Positive semi-definiteness therefore requires:

$$\rho \geq -\frac{1}{n-1}$$

n	**Floor**
2	−1.0000
3	−0.5000
4	−0.3333
5	−0.2500
10	−0.1111

Three assets cannot all be correlated at −0.5, and the bound tightens as n grows. The slider's minimum is recomputed from the live asset count and snapped **up** to the 0.05 step grid, so it can never reach an unrepresentable portfolio. Adding an asset that raises the floor above the current value pulls the slider back up rather than leaving it parked somewhere the engine will reject.

Outside that range σ_p^2 can evaluate negative, and a clamp would silently turn that into a near-zero volatility — collapsing every path onto one deterministic line and reporting VaR 0 on an impossible portfolio. The engine throws `QuantConfigError` instead, surfaced as a message.

II.1.3 · Geometric Brownian Motion path

$$V_{t+1} = V_t \exp\Big[\big(\mu_p - \tfrac{1}{2}\sigma_p^2\big)\Delta t + \sigma_p\sqrt{\Delta t}\; z_t\Big], \qquad z_t \sim \mathcal{N}(0,1), \quad \Delta t = \frac{1}{252}$$

The $-\frac{1}{2}\sigma_p^2$ is the **Itô correction**: without it, $\mathbb{E}[\ln V]$ would drift at μ_p and $\mathbb{E}[V]$ would drift at $\mu_p + \frac{1}{2}\sigma_p^2$, overstating expected wealth.

Example (same portfolio).

$$\mu_p - \tfrac{1}{2}\sigma_p^2 = 0.266000 - 0.085559 = \mathbf{0.180441}$$

$$\text{per-step drift} = \frac{0.180441}{252} = \mathbf{0.00071604}$$

$$\text{per-step } \sigma = 0.413663 \times \sqrt{1/252} = \mathbf{0.02605831}$$

So one step with $z = +1.0$:

$$V_1 = 10{,}000 \times e^{0.00071604 + 0.02605831} = 10{,}000 \times e^{0.02677435} = \mathbf{\$10{,}271.36}$$

> **Horizon units.** The horizon input is in **trading days**, and `periodsPerYear` is pinned to 252 on this tab — stated explicitly at the call site rather than hidden inside the engine. The UI prints the calendar equivalent under the input ("252 → ≈ 1 an") so nobody reads 252 as eight months. The per-asset μ_i, σ_i are genuinely annual (§II.1.0), so the two bases agree.

II.1.4 · Result statistics

Simulated with **seed 42** — the same configuration always produces the same numbers.

Worked example. 1,000 paths, 252 trading days, $10,000 initial, the portfolio above:

Card	Formula	Value
Expected value	$\overline{V_N}$	**$13,001.79**
(sub-line) Expected return	$\overline{V_N}/V_0 - 1$	**+30.02%**
Median	$Q_{0.50}(V_N)$	**$11,762.68**
P5 / P95	$Q_{0.05}, Q_{0.95}$	$6,227.98 * */ * *$**23,841.02**
P(loss)	$\frac{\lVert\{V_N < V_0\}\rVert}{S}$	**32.80%**
(sub-line) P(2×)	$\frac{\lVert\{V_N \geq 2V_0\}\rVert}{S}$	**11.20%**
VaR 95%	$-Q_{0.05}(V_N - V_0)$	**$3,772.02**
CVaR 95%	$-\mathbb{E}[\mathrm{PnL} \mid \mathrm{PnL} \leq -\mathrm{VaR}]$	**$4,796.51**
Best case	$\max V_N$	**$42,720.58**
Worst case	$\min V_N$	**$2,070.12**
(not shown) Std dev	$s(V_N)$	$5,678.49

Note the mean/median gap: $13,002 vs 11{,}763$. GBM produces a **lognormal** terminal distribution, which is right-skewed — the mean is dragged up by a thin tail of very large outcomes. The median is the more representative single number, and P(loss) = 32.8% despite a +30% *expected* return is the same fact stated differently.

Sign convention: Monte Carlo VaR/CVaR are **positive dollar losses**, opposite to the backtest's negative-return convention (§I.1.10).

II.1.5 · Percentile cone (chart)

For each time step *t*, the quantiles are taken **across paths**:

$$\mathrm{cone}_q(t) = Q_q\Big(\{V_t^{(s)}\}_{s=1}^{S}\Big), \qquad q \in \{0.05,\ 0.25,\ 0.50,\ 0.75,\ 0.95\}$$

Paths are capped at 5,000 for the envelope to bound memory; up to 50 individual paths are retained for plotting.

The cone widens as $\sqrt{t}$ — the visual signature of the square-root-of-time rule.

II.1.6 · Final-value histogram (chart)

24 equal-width buckets across the observed range:

$$\text{step} = \frac{\max V_N - \min V_N}{24}, \qquad \text{bucket}(v) = \min\left(23,\ \left\lfloor \frac{v - \min V_N}{\text{step}} \right\rfloor\right)$$

Bars below the initial capital are red, at-or-above are green — so P(loss) is readable directly off the chart.

Degenerate case: if every path lands identically, `step` is 0 and $(v - \min)/0$ is NaN, which would index the bucket array with NaN and throw. One bucket is returned instead.

Example (the run above). step $= (42{,}720.58 - 2{,}070.12)/24 = \$1{,}693.77$. The first bucket is centred at $2,070.12+846.89 = $2{,}917.01$.

II.2 — Tab: Stress Tests

II.2.1 · Shock classes

Code: `src/utils/quantLab.ts:510` — `shockClassFor()`

Holdings are bucketed by **market**, not just asset class:

Class	Contents
`equityDM`	Nasdaq, NYSE — developed markets
`equityEM`	JSE — emerging, globally integrated
`equityFrontier`	BRVM — West African frontier
`bond`	Any `assetClass: "bond"`
`crypto`	Crypto except stablecoins
`stable`	USDT, USDC, DAI, BUSD, TUSD

> **Why equities split three ways.** They did not behave alike in any of these episodes. Lumping them under one shock applied the S&P's −34% COVID drawdown to BRVM listings, which in reality fell by roughly a third of that — the XOF is pegged to the euro, foreign participation is low, and the index is dominated by regional banks and utilities with little exposure to the global cycle.

II.2.2 · The scenario table

Code: `src/utils/quantLab.ts` — `HISTORICAL_SCENARIOS`

Scenario	Window	DM	EM	Frontier	Bond	Crypto	Stable	Recovery
COVID-19 Crash	Feb–Mar 2020	−34%	−35%	−12%	−5%	−50%	0%	150 d
Financial Crisis	Oct 07–Mar 09	−57%	−45%	**n/a**	−12%	**n/a**	0%	517 d
Dot-Com Burst	Mar 00–Oct 02	−49%	−25%	**n/a**	**+20%**	**n/a**	0%	929 d
Luna / FTX	May–Nov 2022	−18%	−10%	0%	−10%	−65%	−2%	190 d
Rate Shock 2022	Jan–Oct 2022	−25%	−15%	−3%	−17%	−65%	0%	282 d
Flash Crash	intraday	−9%	−7%	0%	−1%	−12%	0%	1 d
Geopolitical	hypothetical	−15%	−20%	−8%	−5%	−25%	0%	60 d

> **`null` means "no defensible value", never "zero".** Two distinct cases produce it: the asset class **did not exist** during the episode (Bitcoin's genesis block is January 2009, so crypto shocks for 2008 and 2000 would be fabricated observations, not modelling simplifications); or **we hold no price history** for that market over that window, so a number would be a guess dressed as a measurement. Holdings in a `null` class are **excluded and reported as excluded**, rather than silently shocked by 0 — which would understate losses and read as a deliberate claim that the scenario left them untouched.

Dot-Com's **+20% bond shock** is not a typo: rates fell hard through 2000–2002 and bonds rallied.

`equityFrontier` for 2008/2000 is marked **pending client review** in the source — the open question is whether to source BRVM Composite history for those windows or leave them not-applicable.

II.2.3 · Shock application

Code: `src/utils/quantLab.ts:435` — `runStressTest()`

$$V_i^{\text{after}} = \begin{cases} V_i^{\text{before}} \times (1 + s_c) & \text{if } s_c \text{ is defined} \\ V_i^{\text{before}} & \text{otherwise (excluded)} \end{cases}$$

$$V^{\text{after}} = \text{cash} + \sum_i V_i^{\text{after}}, \qquad \text{PnL}\backslash\% = \frac{V^{\text{after}}}{V^{\text{before}}} - 1$$

Cash is never shocked. Both `undefined` (class absent from the vector) and `null` (explicitly not applicable) mean "this scenario cannot speak to this holding".

Worked example — COVID-19 on the demo portfolio.

Holding	Class	Before	Shock	After	PnL
BTC	crypto	$4,000 \mid -502{,}000$	−$2,000		
ETH	crypto	$2,000 \mid -501{,}000$	−$1,000		
AAPL	equityDM	$1,500 \mid -34990$	−$510		
NVDA	equityDM	$500 \mid -34330$	−$170		
Cash	—	$2,000 \mid — \mid 2{,}000$	$0		
Total		$10,000 \mid\mid$ **6,320**	**−$3,680**		

$$\text{PnL}\backslash\% = \frac{6{,}320}{10{,}000} - 1 = \mathbf{-36.80\%}$$

All seven scenarios on this portfolio:

Scenario	After	PnL	PnL %	Excluded
COVID-19	$6,320 \mid -3{,}680$	−36.80%	—	
Financial Crisis	$8,860 \mid -1{,}140$	−11.40%	**$6,000 (60%)**	
Dot-Com	$9,020 \mid -980$	−9.80%	**$6,000 (60%)**	
Luna / FTX	$5,740 \mid -4{,}260$	−42.60%	—	
Rate Shock 2022	$5,600 \mid -4{,}400$	−44.00%	—	
Flash Crash	$9,100 \mid -900$	−9.00%	—	
Geopolitical	$8,200 \mid -1{,}800$	−18.00%	—	

> **Read the Financial Crisis row carefully.** −11.40% looks mild for the worst equity drawdown in living memory. That is because 60% of the portfolio (BTC + ETH) is **excluded** — Bitcoin did not exist in 2008. The banner states the excluded amount and share explicitly, and the per-asset table shows **n/a** in the Shock column with **—** for PnL, greyed to 60% opacity. The number is not a claim that this portfolio would have lost 11.4% in 2008.

II.2.4 · Exclusion accounting

$$\text{excluded}_{\text{USD}} = \sum_{i\,:\,\neg\text{covered}} V_i^{\text{before}}, \qquad \text{excluded}_{\text{share}} = \frac{\text{excluded}_{\text{USD}}}{V^{\text{before}}}$$

Example. Portfolio of BTC $4,000 + $AAPL1,500 + cash $2,000 =7,500, under the Financial Crisis scenario:

$$\text{excluded} = \$4{,}000, \qquad \text{share} = \frac{4{,}000}{7{,}500} = \mathbf{53.33\%}$$

$$V^{\text{after}} = 2{,}000 + 4{,}000 + 1{,}500(1 - 0.57) = 2{,}000 + 4{,}000 + 645 = \mathbf{\$6{,}645}$$

Note the share denominator is the **full** portfolio value including cash.

II.2.5 · Recovery path

Code: `src/utils/quantLab.ts:420` — `recoveryPath()`

Linear interpolation from the post-shock trough back to the pre-shock value over the scenario's observed recovery window:

$$V(t) = V^{\text{trough}} + \left(V^{\text{pre}} - V^{\text{trough}}\right)\frac{t}{T_{\text{rec}}}$$

Sampled at $\min(30, \max(2, \lceil T_{\text{rec}}/7 \rceil))$ points — roughly weekly, capped at 30.

> **Linear is deliberately the simplest defensible shape.** Real recoveries are not linear. This is presented as a **horizon**, not a forecast: an instantaneous shock with no time dimension tells you the drawdown but not how long capital stays impaired, which is usually the question that matters for a portfolio with obligations to meet.

Example — COVID-19, $T_{\text{rec}} = 150$ days. $\lceil 150/7 \rceil = 22$ intervals → 23 points:

Point	Day	Value
0	0	$6,320.00
1	7	$6,487.27
2	14	$6,654.55
3	20	$6,821.82
…	…	…
22	150	$10,000.00

Per-step increment: $(10{,}000 - 6{,}320)/22 = \$167.27$.

II.2.6 · Custom scenario

Code: `src/utils/quantLab.ts:485` — `runCustomStress()`

Three sliders (equities / crypto / stablecoins), each −90% to +50% in 1% steps. Bonds default to 0%, recovery to 30 days.

$$s_{\text{equityDM}} = s_{\text{equityEM}} = s_{\text{equityFrontier}} = \text{equityPct}$$

The single equity input applies to all three equity buckets. A hand-set scenario carries no basis for differentiating them, and inventing a regional split the user did not ask for would misrepresent their input.

Example. Equities −20%, crypto −40%, stables 0%, on the demo portfolio:

$$V^{\text{after}} = 2{,}000 + 4{,}000(0.6) + 2{,}000(0.6) + 1{,}500(0.8) + 500(0.8)$$
$$= 2{,}000 + 2{,}400 + 1{,}200 + 1{,}200 + 400 = \mathbf{\$7{,}200} \to \mathbf{-28.00\%}$$

II.2.7 · All-scenarios comparison (chart)

Runs all seven scenarios against the same portfolio and bar-charts $\text{PnL}\% \times 100$. Green for ≥ 0, red otherwise.

Axis labels come from the scenario **id**, not from the first word of the display name — an earlier version used `scenarioName.split(" ")[0]`, which rendered "Crise" / "Éclatement" in every locale regardless of language.

II.3 — Tab: VaR / CVaR

Parametric (Gaussian) risk, in dollars, over a chosen horizon. Structurally different from both §I.1.10 (historical, single-period, in return units) and §II.1.4 (simulated, in dollars).

II.3.1 · Horizon volatility

Code: `src/utils/quantLab.ts:538` — `horizonVolatility()`

$$\sigma_h = \frac{\sigma_{\text{annual}}}{\sqrt{P}} \sqrt{\max(1, h)}$$

Square-root-of-time from annual to h trading days, with $P = 252$ on this tab.

Example. $\sigma_{\text{annual}} = 40\%$, $h = 10$ days:

$$\sigma_h = \frac{0.40}{\sqrt{252}} \sqrt{10} = 0.0251976 \times 3.162278 = \mathbf{0.07968191} \to \mathbf{7.97\%}$$

> Assumes i.i.d. returns. Real returns exhibit **volatility clustering**, so this understates risk at longer horizons.

II.3.2 · Parametric VaR

Code: `src/utils/quantLab.ts:550` — `parametricVaR()`

$$\text{VaR}_\alpha = V \cdot z_\alpha \cdot \sigma_h$$

Returned as a **positive** loss figure. Assumes **zero drift** — standard at short horizons, conservative at long ones.

Example. $V = \$10{,}000$, $\sigma_{\text{annual}} = 40$, $h = 10$, $\alpha = 95$:

$$\text{VaR} = 10{,}000 \times 1.644854 \times 0.07968191 = \mathbf{\$1{,}310.65}$$

The card's sub-line shows $1{,}310.65 / 10{,}000 = \mathbf{13.11\%}$ of capital.

II.3.3 · Parametric CVaR (expected shortfall)

Code: `src/utils/quantLab.ts:565` — `parametricCVaR()`

$$\mathrm{CVaR}_\alpha = V \cdot \sigma_h \cdot \frac{\phi(z_\alpha)}{1-\alpha}$$

Example. Same inputs:

$$\mathrm{CVaR} = 10{,}000 \times 0.07968191 \times \frac{0.103136}{0.05} = 10{,}000 \times 0.07968191 \times 2.06272 = \mathbf{\$1{,}643.61}$$

Ratio to VaR: $1{,}643.61 / 1{,}310.65 = 1.254$. Under normality that ratio depends **only** on α, never on V or σ — it is $\phi(z)/\left[(1-\alpha)z\right]$.

> **Why `normInv` had to be a real function.** This formula divides by $\phi(z)$, which is highly sensitive to z. An earlier four-value lookup table returned the 97.5% z-score for a 98% request (1.96 instead of 2.054) — a 4.6% error in z becomes a **26%** error in $\phi(z)$ and propagates straight into CVaR.

II.3.4 · Portfolio prefill ("Use my portfolio")

Code: `src/routes/_authenticated/quant-lab.tsx` — `portfolioEstimate`

Value-weighted blend of each real position's own realized volatility:

$$V = \sum_{i \in \text{covered}} V_i + \text{cash}, \qquad \sigma = \frac{\sum_{i \in \text{covered}} V_i \, \sigma_i}{V}$$

Three correctness rules encoded here:

1. **Cash belongs in both bases or neither.** Volatility is a property of the whole portfolio the value refers to. Dividing a risky-only volatility by a risky-only total and then applying it to a cash-inclusive value **overstates VaR by the cash ratio** — a portfolio that is 90% cash would inflate it roughly tenfold. Cash contributes 0 to the numerator and its full value to the denominator, which is what dilutes portfolio volatility correctly.
2. **Positions with under 10 bars of history are excluded from both** numerator and covered value, and reported separately in a notice. Counting them at zero volatility would drag the estimate down by exactly the share we know least about.
3. Ignores cross-asset correlation, consistent with this tab's single-volatility-input design. This **understates** portfolio volatility for any positively-correlated book.

Example. BTC $6,000 (\sigma 602,000 ($\sigma$ 25%), cash $2,000, all covered:

$$V = 6{,}000 + 2{,}000 + 2{,}000 = \$10{,}000$$

$$\sigma = \frac{6{,}000(0.60) + 2{,}000(0.25)}{10{,}000} = \frac{3{,}600 + 500}{10{,}000} = \mathbf{41.00\%}$$

The button rounds these into the Portfolio value and Annual volatility fields.

II.3.5 · VaR / CVaR term structure (chart)

Both measures recomputed for $h = 1 \ldots 30$:

$$\mathrm{VaR}(h) = V z_\alpha \frac{\sigma}{\sqrt{P}} \sqrt{h} \;\propto\; \sqrt{h}$$

Both curves are pure $\sqrt{h}$ shapes — the badge reads "√t scaling" to say so.

Example ($V = \$10{,}000$, $\sigma = 40$, 95%):

h	VaR	CVaR
1	414	520
4	829	$1{,}039$
10	$1{,}311$	$1{,}644$
30	$2{,}270$	$2{,}847$

Quadrupling the horizon exactly doubles the risk — which is the assumption, not an observation.

II.3.6 · Reading these numbers honestly

The Gaussian assumption is the weak link, and the engine says so in the explainer card. Two specific failure modes:

- **Excess kurtosis** (§I.2.10) means real tails are fatter than normal. For crypto, empirical excess kurtosis routinely exceeds +10, and Gaussian VaR **materially understates** the tail.
- **Zero drift** is assumed. Over a 252-day horizon on a high-drift asset that is a real omission, though it errs conservative for a positive-drift portfolio.

Cross-check any Gaussian VaR here against the **historical** VaR on the Backtesting Lab (§I.1.10), which makes no distributional assumption. A large gap between the two is the kurtosis showing up.

Appendix A — Reference datasets

Every worked example above uses one of these three.

Dataset E — equity curve (11 bars, $10,000 start$, P = 252$)

```
Equity:  10000, 10200, 10098, 10502, 10187, 10391, 9975, 10175, 9971, 10370, 10162
Returns: 0.020000, -0.010000,  0.040008, -0.029994,  0.020026,
        -0.040035,  0.020050, -0.020049,  0.040016, -0.020058
```

Statistic	Value
mean(r)	0.001996
stdev(r)	0.029386
Total return	1.6200%
CAGR	49.9255%
Annual volatility	46.6488%
Sharpe	1.078453
Sortino	1.717011
Calmar	9.874161
Max drawdown	−5.0562%
Average drawdown	−2.0672%
VaR 95%	−3.5517%
CVaR 95%	−4.0035%
Skewness	−0.010638
Excess kurtosis	−1.772170
Best / Worst day	+4.0016% / −4.0035%

Dataset B — benchmark (11 bars, paired with E)

```
Prices:  10000, 10300, 10094, 10598, 10174, 10479, 9955, 10254, 9946, 10444, 10131
Returns: 0.030000, -0.020000,  0.049931, -0.040008,  0.029978,
        -0.050005,  0.030035, -0.030037,  0.050070, -0.029969
```

Statistic (E vs B)	Value
Beta	0.742645
Alpha (annualized)	+12.8872%
Tracking error	16.6995%
Information ratio	−0.004850

Dataset R — three assets, 10 daily returns each

```
A: 0.010, -0.020,  0.030, -0.010,  0.020,  0.000, -0.015,  0.025,  0.005, -0.005
B: 0.020, -0.030,  0.050, -0.020,  0.040,  0.010, -0.020,  0.030,  0.000, -0.010
C: 0.003,  0.001, -0.002,  0.004,  0.000,  0.002,  0.001, -0.001,  0.003,  0.002
```

	A	B	C
Annualized μ	100.80%	176.40%	32.76%
Annualized σ	27.19%	43.67%	3.00%

Correlations: $\rho_{AB} = 0.9717$, $\rho_{AC} = -0.6080$, $\rho_{BC} = -0.6224$. Shrinkage: $\delta = 0.60$ ($n = 3$, $T = 10$).

Optimizer outputs on the shrunk matrix:

Strategy	A	B	C	Portfolio σ
Equal weight	0.3333	0.3333	0.3333	19.64%
Min variance	0.0274	0.0137	0.9588	2.76%
Max Sharpe	0.0326	0.0398	0.9276	— (Sharpe 13.47)
Risk parity (ERC)	0.0740	0.0463	0.8797	3.61%
Vol target 12%, $L_{\max}$ 1.5	0.1110	0.0694	1.3196	5.41% (cash −50%)
Vol target 2%, $L_{\max}$ 1.5	0.0410	0.0257	0.4877	2.00% (cash +44.56%)

Appendix B — Reading a real result card

From an actual run: AAPL 40% / BTC 40% / BICB 20%, 1 Y, $10,000, Buy & Hold, monthly rebalance, BTC benchmark, 10 bps fees, 5 bps slippage, 0% risk-free.

Card	Value
Total return	−11.54%
CAGR	−16.82%
Alpha	+7.00%
Beta	0.61
Sharpe	−0.50
Sortino	−0.67
Calmar	−0.73
Max drawdown	−22.93%
Annual volatility	28.80%
VaR 95%	−2.87%
CVaR 95%	−4.18%
Win rate	44.44%
Final equity	$8,833

Four consistency checks you can do by hand:

1. Total return vs final equity — and why $10,000 is the wrong denominator.

$$\frac{8{,}833}{10{,}000} - 1 = -11.67\% \neq -11.54\% \quad ✗$$

The opening trade is charged (§I.0.4): $c = 1.00 \times (0.0010 + 0.0005) = 0.0015$, so $V_0 = 10{,}000 \times 0.9985 = \$9{,}985$.

$$\frac{8{,}833}{9{,}985} - 1 = \mathbf{-11.54\%} \quad ✓$$

That $15 gap is the **entire** reconciliation. It is also the first entry in Fees Paid (§I.2.24).

2. Calmar.

$$\frac{-16.82\%}{|-22.93\%|} = \mathbf{-0.7335} \to -0.73 \quad \checkmark$$

3. Sharpe implies the arithmetic mean. With $R_f = 0$:

$$\bar{r} \cdot P = S \times \sigma_p = -0.50 \times 0.2880 = \mathbf{-14.40\%}$$

Note CAGR (−16.82%) sits **below** this. That is expected, not an error: the geometric mean is always below the arithmetic mean, by roughly $\frac{1}{2}\sigma^2 = \frac{1}{2}(0.288)^2 = 4.15$ — and -14.40, the right order of magnitude for the gap. Volatility drag is real money.

4. Alpha implies the benchmark's return.

$$7.00\% = -14.40\% - 0.61 \times \bar{R_m} \implies \bar{R_m} \approx -35.1\%$$

Consistent with the benchmark line on the chart, which ends near $7,400 from 10{,}000$ over about eight months. **The strategy lost 11.5% and still produced +7% alpha**, because at $\beta = 0.61$ it was only expected to capture 61% of a −35% benchmark — i.e. −21.4% — and it lost less than that.

5. Tail shape. CVaR/VaR $= 4.18 / 2.87 = 1.456$. Under normality at 95% that ratio is fixed at **1.254** (§II.3.3). The realized 1.456 is materially higher, which says the left tail of this strategy is fatter than Gaussian — exactly the situation in which the Quant Lab's parametric VaR (§II.3.2) would understate risk.

Footnote on the config screenshot. That run reported *"No historical data for: BICB"*. BICB is a BRVM listing, and at the time only crypto history had been seeded. The engine refuses to run rather than silently dropping the asset and reporting metrics for a 2-asset portfolio the user did not configure.

Cross-reference: metric appears in more than one place

Metric	Backtest Results	Backtest Advanced	Backtest Models	Quant Lab	Same formula?
Total return	§I.1.1	§I.2.1	—	—	Yes
CAGR	§I.1.2	§I.2.2	—	—	Yes
Annual volatility	§I.1.9	§I.2.3	§I.5.4	§II.3.4 (input)	Yes
Sharpe	§I.1.5	§I.2.4	§I.5.5	—	Yes
Sortino	§I.1.6	§I.2.5	§I.5.6	—	Yes
Calmar	§I.1.7	§I.2.6	—	—	Yes
Max drawdown	§I.1.8	§I.2.7	§I.5.7	—	Yes
VaR 95%	§I.1.10	§I.2.11	§I.5.8	§II.1.4, §II.3.2	**No** — historical vs simulated vs parametric
CVaR 95%	§I.1.11	§I.2.12	§I.5.9	§II.1.4, §II.3.3	**No** — same three variants
Beta	§I.1.4	§I.2.13	§I.5.10	—	Yes
Alpha	§I.1.3	§I.2.14	§I.5.11	—	Yes
Win rate	§I.1.12	§I.2.17	—	—	Yes
Profit factor	—	§I.2.18	§I.4.1 (tile 6)	—	**No** — differs when there are no losers
Avg win / loss	—	§I.2.19–20	§I.4.1 (tiles 4–5)	—	Same value; differs only at zero count
Volatility (portfolio)	§I.1.9 (realized)	—	§I.5.4	§II.1.1 (quadratic form)	**No** — series stdev vs $\sqrt{w'\Sigma w}$
Correlation	—	—	§I.5.16 (estimated)	§II.1.2 (user slider)	**No** — estimated vs assumed

Known deviations and open items

Recorded so they are not mistaken for oversights.

1. **Exposure is not computed** (§I.2.25), though two strategies now have a real cash leg that would make it meaningful.
2. **`equityFrontier` shocks for 2008 and 2000 are `null`**, pending a decision on sourcing BRVM Composite history for those windows (§II.2.2).
3. **Profit factor differs between §I.2.18 and §I.4.1 tile 6** when there are no losing trades (`null` → "—" vs `0.00`). The Advanced metrics row is correct.
4. **Monte Carlo assumes constant correlation and lognormal terminals.** It does not use the estimated covariance matrix of §I.5.16 — correlation is the user's slider. The two tabs are deliberately not wired together.
5. **Quant Lab pins `periodsPerYear = 252`** on both the Monte Carlo and VaR tabs, while the Backtesting Lab infers it (§0.3). Per-asset drift/vol are still calibrated on each asset's own inferred cadence, so the annual figures are consistent; only the step basis is fixed.